

From analysis to design

From Analysis to Design, System Sequence Diagram (SSD) and
Operation Contracts (OC)

New Groups in P2

This term due to multiple factors, we will be creating new teams, they will be created during or before the first class of P2 by the lecturers.

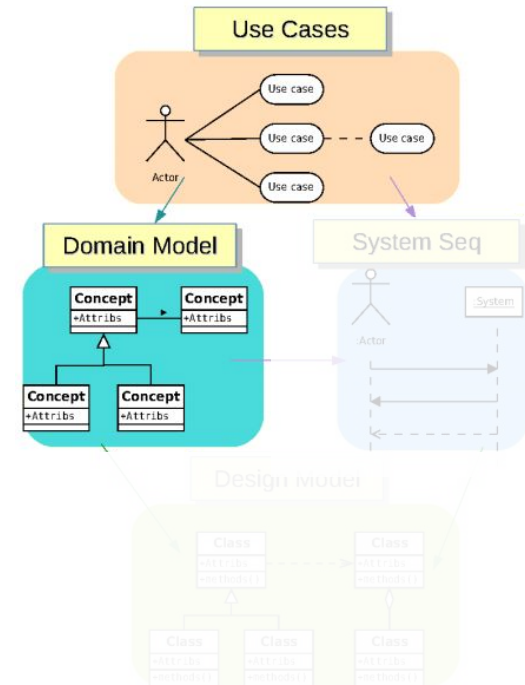
The new teams will be consisting of **3 members** each.

When the groups are made you will be asked to rank your preferred cases and a case will be assigned to you by the lecturer to avoid everyone working on the same case.

SSD: System Sequence Diagram

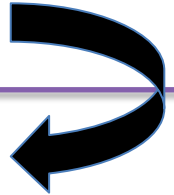
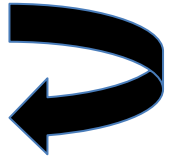
The Road to Design

- During the analysis phase we analyze
 - System Behavior: Use Cases
 - The structure of the system: Domain Model
- We now convert the above models into a System Sequence Diagram
- The Conceptual Class Diagram together with the System Sequence Diagram (and contracts) then provides all the necessary information to develop a Design of the system.



Conceptual Thinking: Analysis -> Design

	STATIC / STRUCTURE	DYNAMIC / BEHAVIOR / INTERACTION
ANALYSIS	DOMAIN MODEL = Conceptual Class Diagram Reading direction multiplicities	Use Cases descriptions Use Case Diagram System Sequence Diagram Operation Contracts
DESIGN		



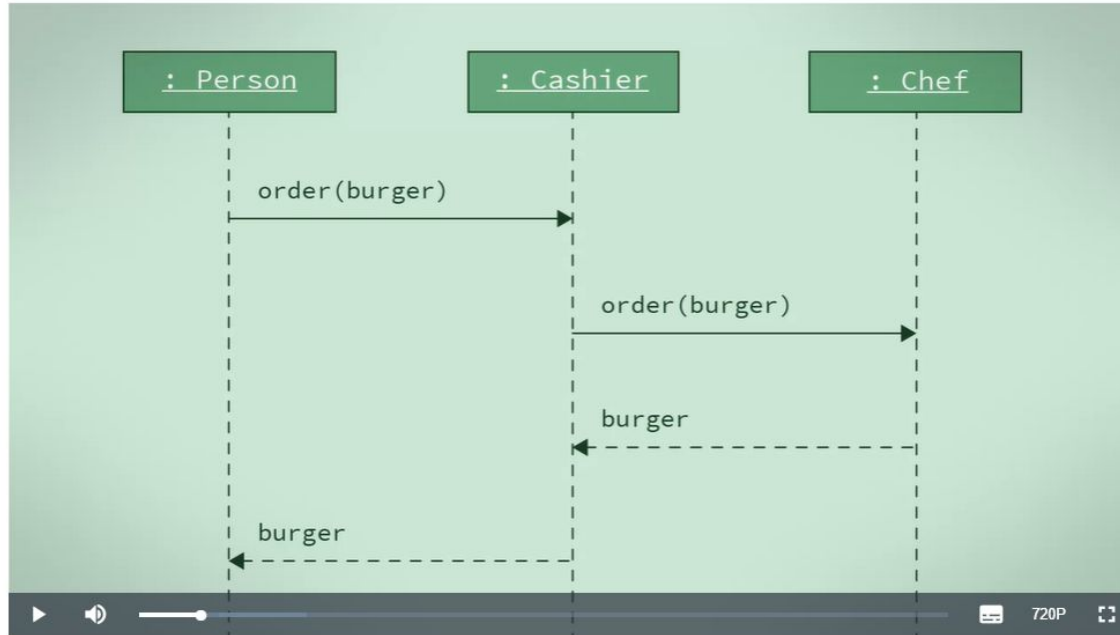
System Sequence Diagram (SSD)

- Very closely related to analysis: use case descriptions and domain model
- Models the interactions described in text form in use case descriptions
 - So don't draw all the steps in HSS and alternative scenarios, only those that represent communication between actor and system
- Display commands we give to the system (not commands to actors!): solid line
- Provide possible responses from system to actor: dotted line

Knowledge Clip: 7 minutes (Ordering a burger)

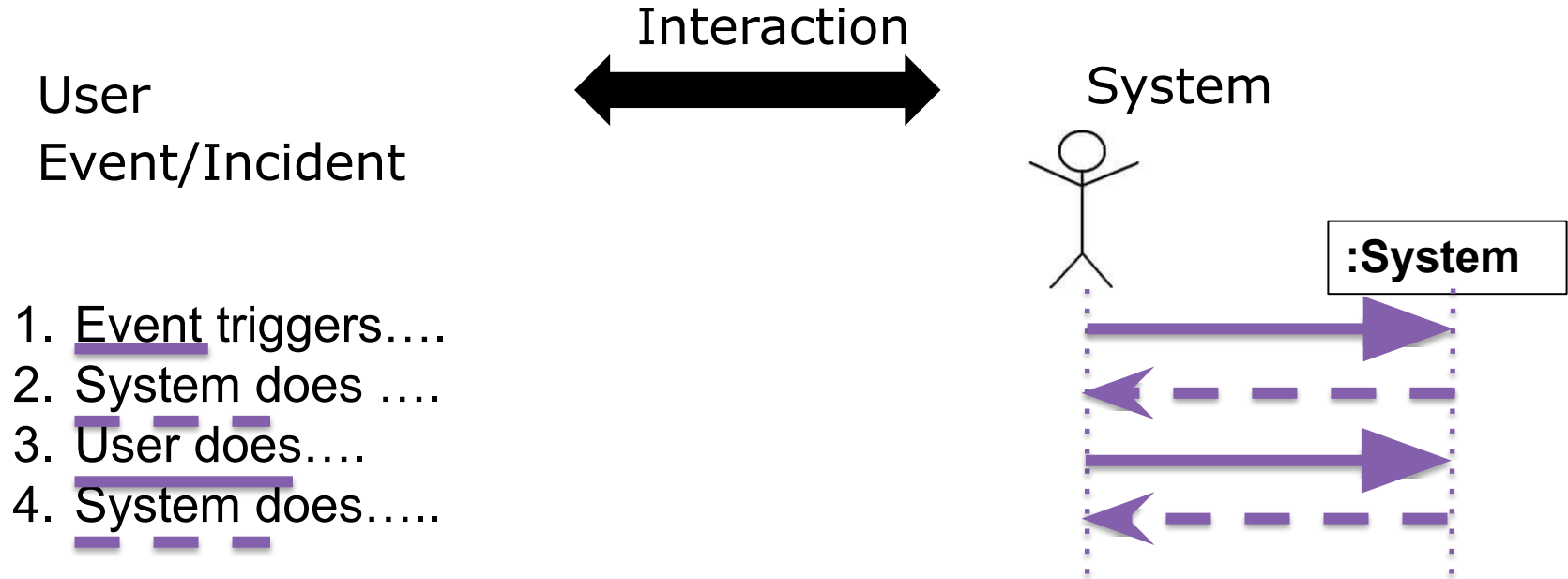
1.3.6 – UML Sequence Diagram

[Share](#)



<https://www.coursera.org/lecture/object-oriented-design/1-3-6-uml-sequence-diagram-965yb>

System Sequence Diagram (SSD): What & How?



System Sequence Diagram (SSD): What & How?

Use Case Scenario



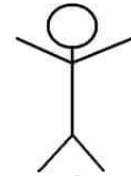
System Sequence Diagram

- 1. Event triggers....
- 2. System does
- 3. User does....
- 4. System does.....

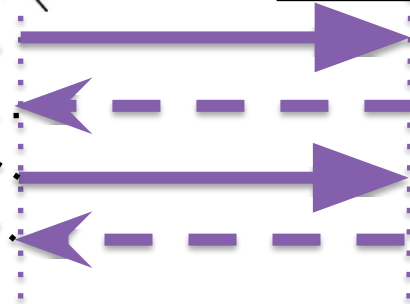
Interactions:

Input event

Output event



:System

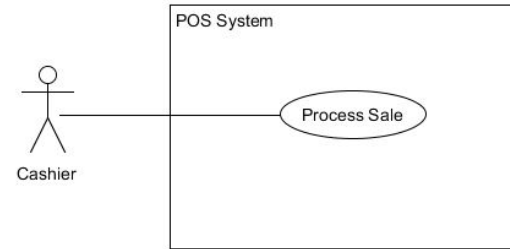


What is System Sequence Diagram (SSD)?

Simple cash-only Process Sale scenario:

1. Customer arrives at a POS checkout with goods and/or services to purchase.
2. Cashier starts a new sale.
3. Cashier enters item identifier.
4. System records sale line item and presents item description, price, and running total.
Cashier repeats steps 3-4 until indicates done.
5. System presents total with taxes calculated.
6. Cashier tells Customer the total, and asks for payment.
7. Customer pays and System handles payment.

...

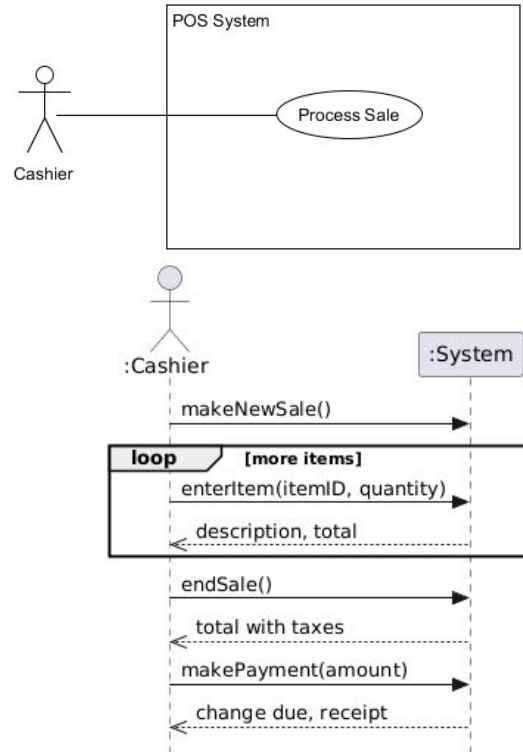


What is System Sequence Diagram (SSD)?

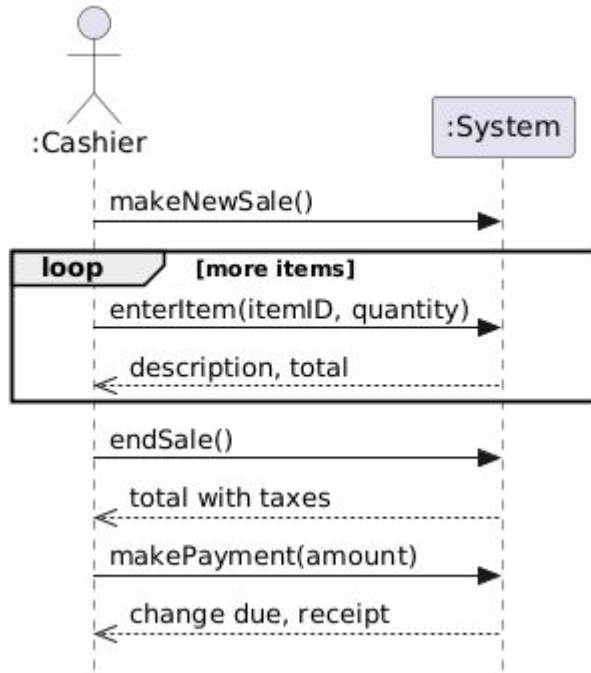
Simple cash-only Process Sale scenario:

1. Customer arrives at a POS checkout with goods and/or services to purchase.
 2. Cashier starts a new sale.
 3. Cashier enters item identifier.
 4. System records sale line item and presents item description, price, and running total.
- Cashier repeats steps 3-4 until indicates done.
5. System presents total with taxes calculated.
 6. Cashier tells Customer the total, and asks for payment.
 7. Customer pays and System handles payment.

...

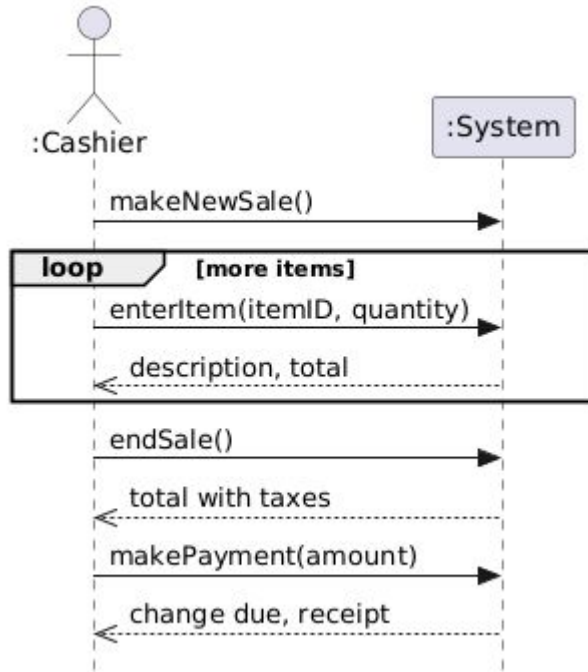


What is System Sequence Diagram (SSD)?



- Simplest kind of Sequence Diagram
- **System** is considered as one class (ie. **black box**)
- Shows interaction between actor and system
- Capturing which events the system should respond to
 - External events from people (actor) or other systems
 - Timerevents
 - Exceptions
- Starting point for the implementation of behavior
- *:System is notation from Object Diagram for an anonymous instance of the System class*

Why System Sequence Diagram SSD?



- Part of Use Case (Analysis)
- Especially in **the main scenario of most important use cases**
- Can also be used for other commonly used or complex scenarios
- Rarely used as permanent documentation in a project
- Is also used for
 - Risk hedging: getting a clear picture of the largest operations
 - To add Operation Contracts
 - To make estimates

Method: How to create an SSD?

1. Actors determine or adopt from the use case

- One primary actor, :Actor name, on the left
- :System as black box on the right
- possibly several supporting actors more to the right
- Use Case title at top

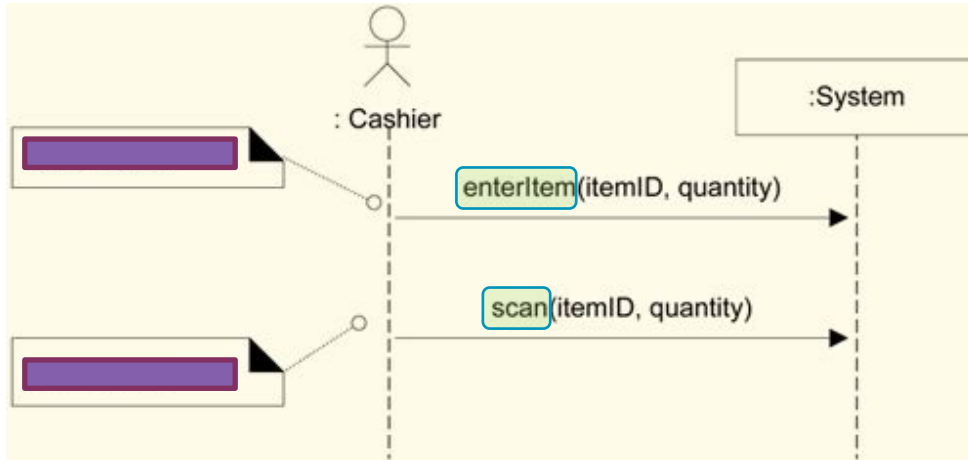
2. Determining what interactions exist between actors and the system

1. Customer arrives at POS checkout with goods and/or services to purchase.
2. Cashier starts a new sale.
3. Cashier enters item identifier.
4. System records sale line item and presents item description, price, and running total. Price calculated from a set of price rules.
Cashier repeats steps 3-4 until indicates done.
5. System presents total with taxes calculated.
6. Cashier tells Customer the total, and asks for payment.
7. Customer pays and System handles payment.
8. System logs completed sale and sends sale and payment information to the external Accounting system (for accounting and commissions) and Inventory system (to update inventory).
9. System presents receipt.
10. Customer leaves with receipt and goods (if any).

How to create an SSD?

3. Naming input messages

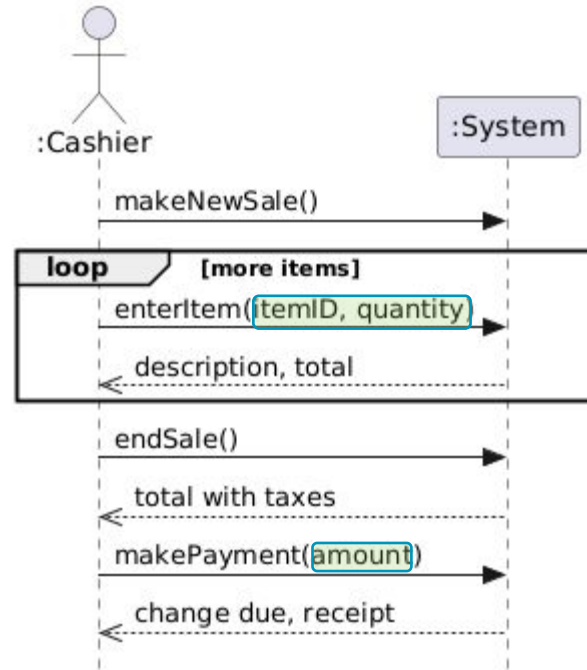
- As an abstract description of the event's purpose. Phrase it in the imperative, as a command.
- **Not in terms of solution (this is Design rather than Analysis)**
 - Which of the two, scan or enterItem, is better?



How to create an SSD?

4. Determining parameters on input messages

- What data do we need to provide so that the system can perform this operation?
- The data can be found as attributes in the domain model. All data should also be available there. Alignment between the models!

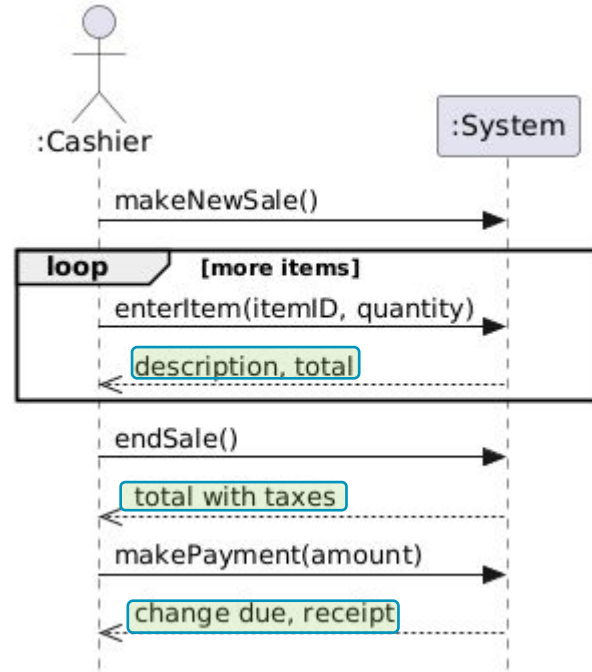


How to create an SSD?

5. Determine answers/return messages

- Does the system respond to an input message?
 - Is not mandatory
- So, what data is returned?

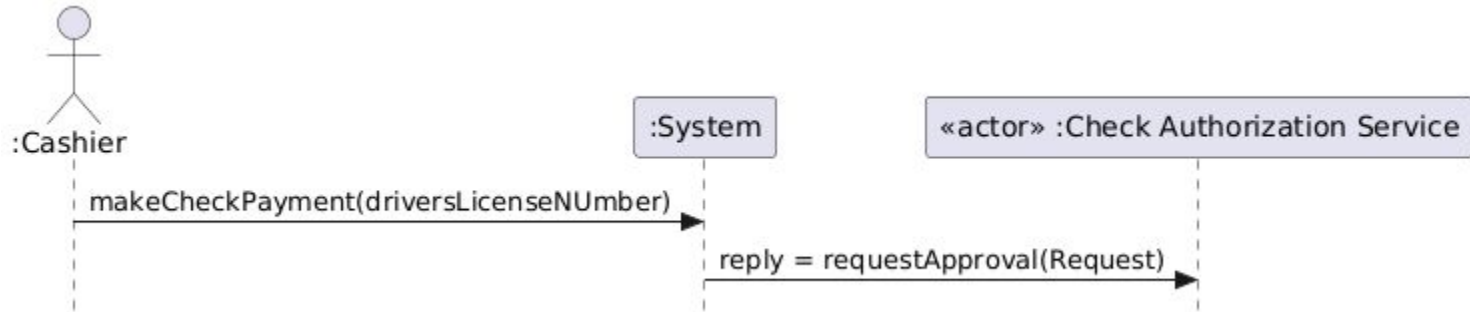
Note the shape: dashed line with an arrow in the shape of a check mark



What to do the secondary actors?

- Does the Use Case have interactions with other actors (secondary/external)?

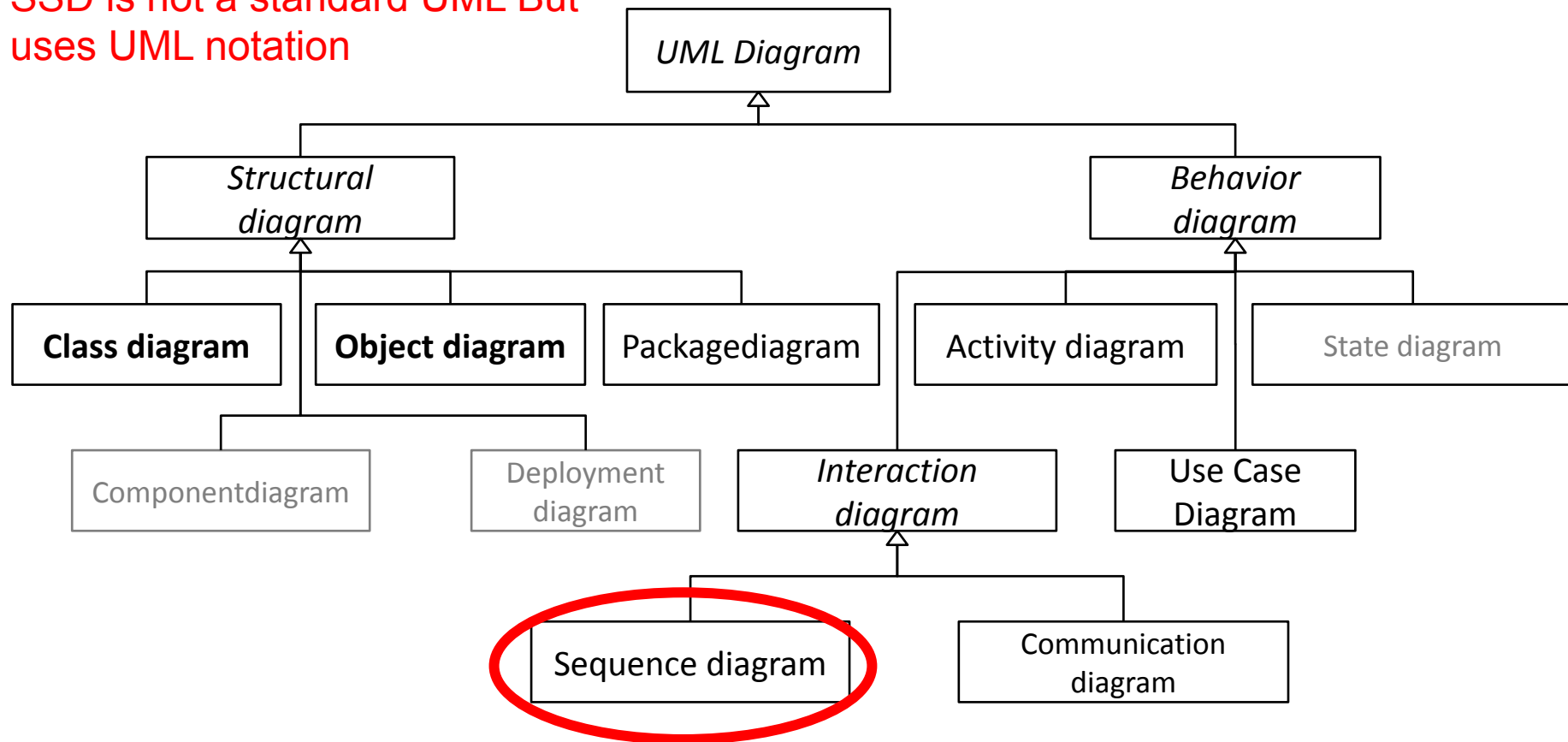
If any, draw them on the right side!



Secondary actor
Position: right (as in UC diagram!)

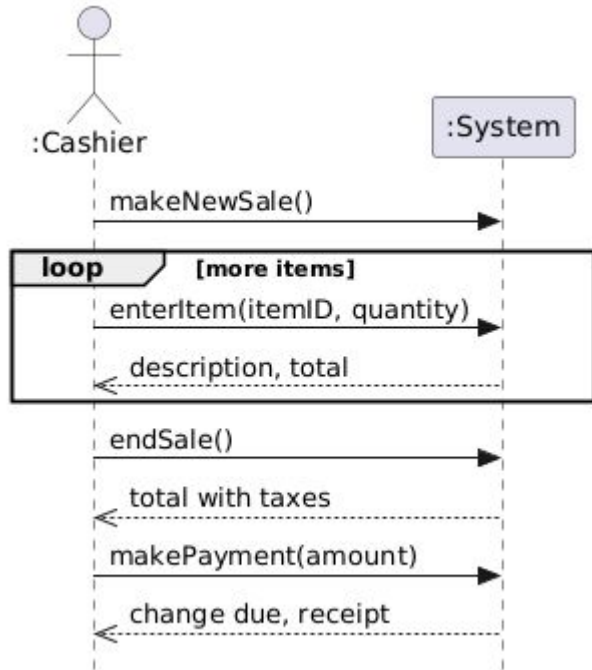
Is an SSD a UML diagram?

SSD is not a standard UML But
uses UML notation

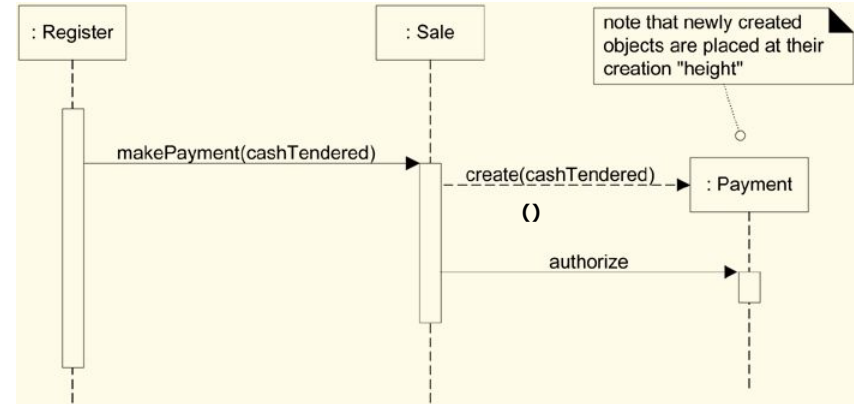


Moving ahead... Spot the differences.

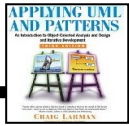
System Sequence diagram



Interaction diagram
Sequence Diagram

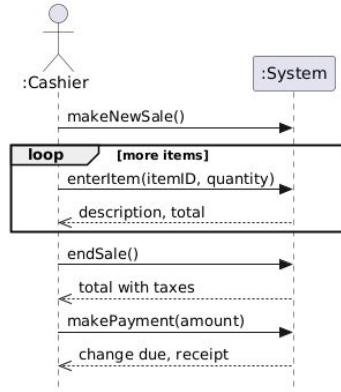


Moving ahead... Spot the differences.



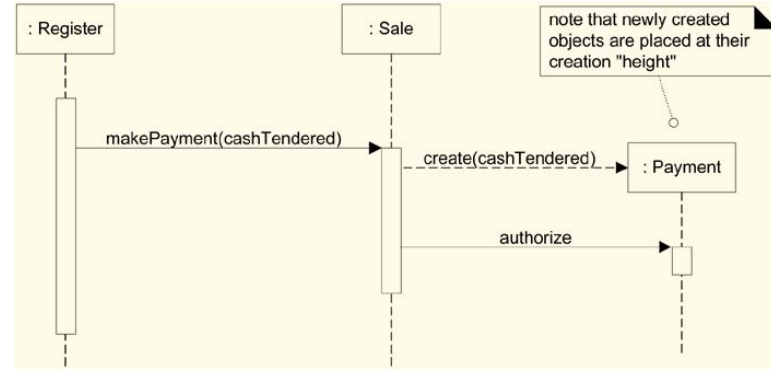
10.2
10.4

System Sequence diagram



- Interactions between actor and system
- explicit return data
- Consider system as a class (black box principle)
- SSD is analysis of the problem (what?)
 - Return messages to better display output, multiple return items possible (analysis)
 - no lifeline activation

Interaction diagram Sequence Diagram



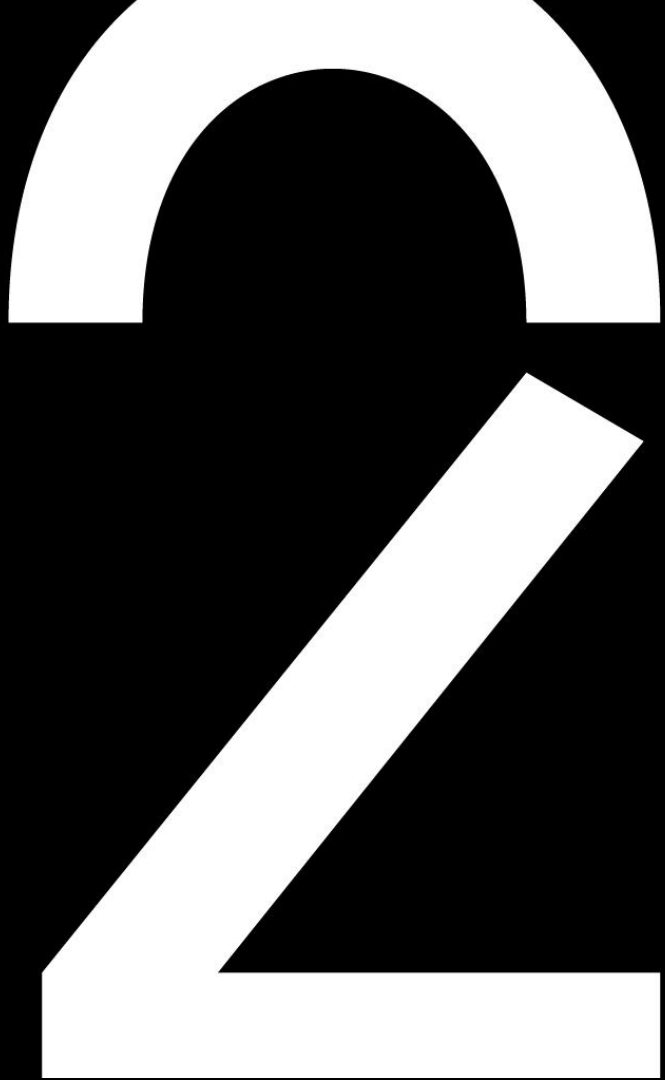
- Interactions between objects
- activation box lifeline
- Instances of class
- SD is implementation (how?)
- SD is the implementation
 - Interaction message from SSD becomes (incoming) message in SD

Recurring Example: Monopoly

Check end of slide deck

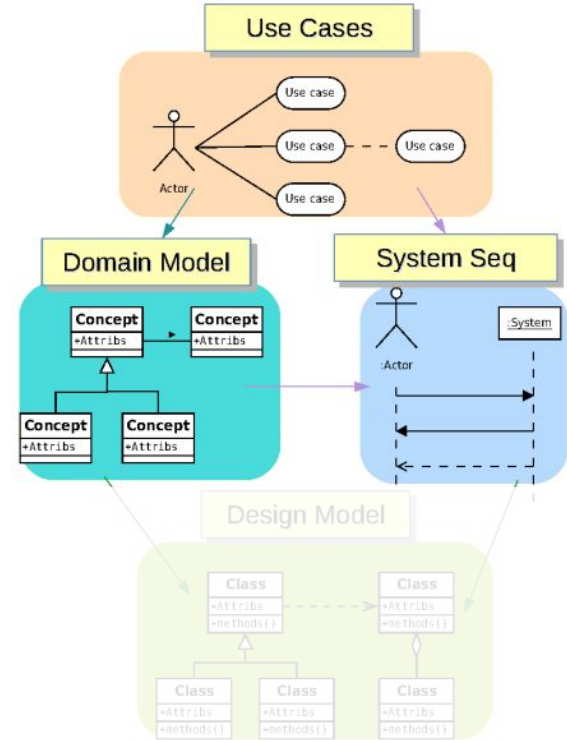


Operation Contracts



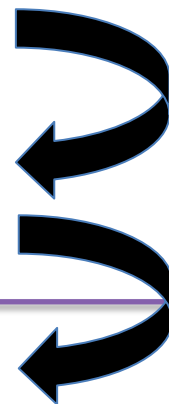
The Road to Design

- We know by now
 - System **Behavior**: Use Cases
 - The **structure** of the system: Domain Model
 - A **visual** summary of 1 Use Case : SSD
- **Sometimes** we need **more detail** to understand the SSD better and to be used in the next stage of design!
- For that we use **Operation Contracts (OC's)**.
- **Combined** with the **SSD**, the **OC's** determine the **final** requirements before we actually go into 'Design' mode



Conceptual Thinking: Analysis -> Design

	STATIC / STRUCTURE	DYNAMIC / BEHAVIOR / INTERACTION
ANALYSIS	DOMAIN MODEL = Conceptual Class Diagram Reading direction multiplicities	Use Cases descriptions Use Case Diagram
DESIGN		System Sequence Diagram Operation Contracts



Operation Contracts

- **What do we know by now?**

UC's and SSD's show the external system behavior.

- **What do we miss?**

How the operation calls **affect/change** the internal states of the system.

"What is happening with the objects in the domain model?"

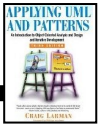
⇒ **This is what we do in the OC's!**

- An **OC** specifies **what and how the internal state** of the system changes as a result of the execution of ONE operation (from the SSD)
- We observe at the **aftermath** (consequences) of the **event** on the **domain model**

Operation Contracts

- INPUT :
 - **Concepts** and **associations** from the Domain Model
 - System **events** shown in the SSD
- OUTPUT :
 - **Mini-contract** for each operation with the **main focus** on **POST-condition** description!

Example: What does an Operation Contract look like?



11.1

Name of an input message from the SSD

Name of the UC to which the operation belongs

System situation before the operation was started. $\Leftrightarrow$ UC preconditions are process conditions

System-technical changes resulting from the operation. There are three types of changes.
 $\Leftrightarrow$ UC postconditions are process results

Contract CO1 : enterItem

Operation

enterItem(itemID,quantity)

Cross-reference

Use Case(s) Process Sale

Preconditions

Retrieve instance within scope of UC

- Sale agency was present
- Instance pd of ProductDescription was present (via argument)

Postconditions

Instance creations and deletions

- Instance sli of SalesLineItem was created

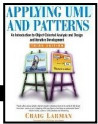
Link creations and deletions

- Instance sli of SalesLineItem was linked to instance s of Sale
- Instance sli of SalesLineItem was linked with instance pd of ProductDescription

Changes in attribute values

- sli.quantity was given the value quantity (via argument)

Example: How does an Operation Contract look like?



11.1

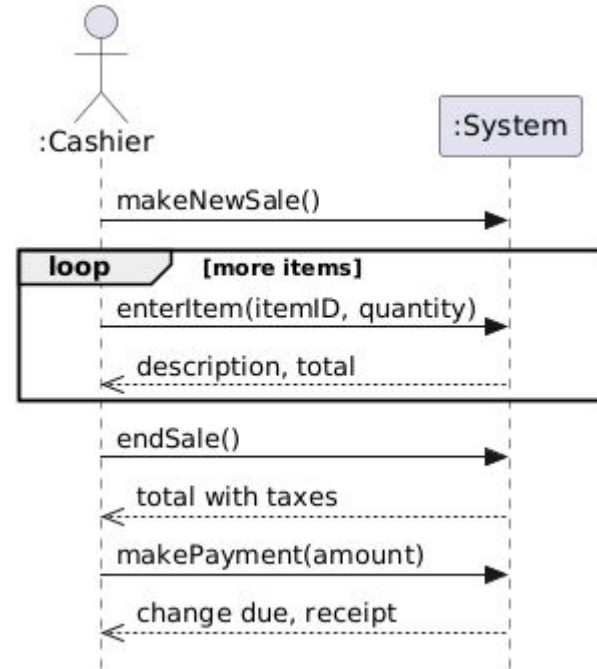
Contract CO1 : enterItem		
Operation	enterItem(itemID,quantity)	Matches the corresponding input events in the SSD - arrows to the right, no 'attribute type' needed yet.
Cross-reference	Use Case(s) Process Sale	What Use Case is this?
Preconditions	• Sale agency was present • Instance pd of ProductDescription was present (via argument)	Especially nontrivial assumptions about the status of objects at the start of the operation – i.e., sometimes empty.
Postconditions	• Instance sli of SalesLineItem was created	The most important section! Describes the state of the objects in the Domain Model AFTER the operation
	• Instance sli of SalesLineItem was linked to instance s of Sale • Instance sli of SalesLineItem was linked with instance pd of ProductDescription	
	• sli.quantity was given the value quantitiy (via argument)	

Example: NextGenPos Input: SSD for Process Sale

Simple cash-only Process Sale scenario:

1. Customer arrives at a POS checkout with goods and/or services to purchase.
 2. Cashier starts a new sale.
 3. Cashier enters item identifier.
 4. System records sale line item and presents item description, price, and running total.
- Cashier repeats steps 3-4 until indicates done.
5. System presents total with taxes calculated.
 6. Cashier tells Customer the total, and asks for payment.
 7. Customer pays and System handles payment.

...



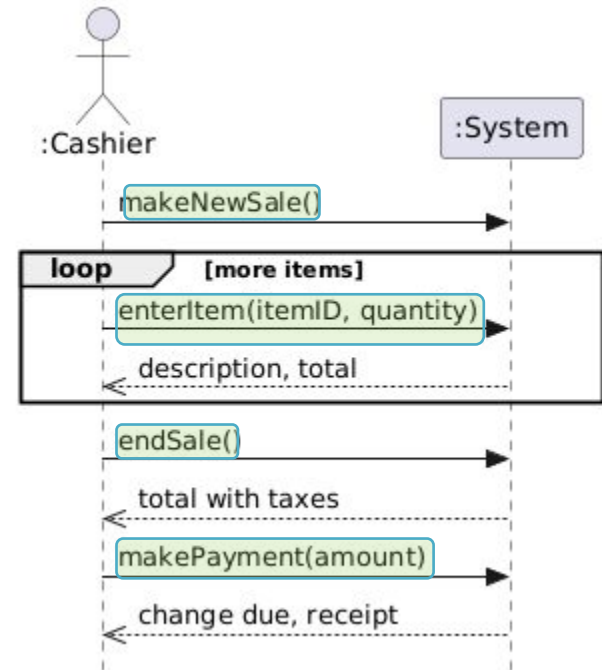
Input: What operations are needed?

Example: NextGenPos Input: SSD for Process Sale

Simple cash-only Process Sale scenario:

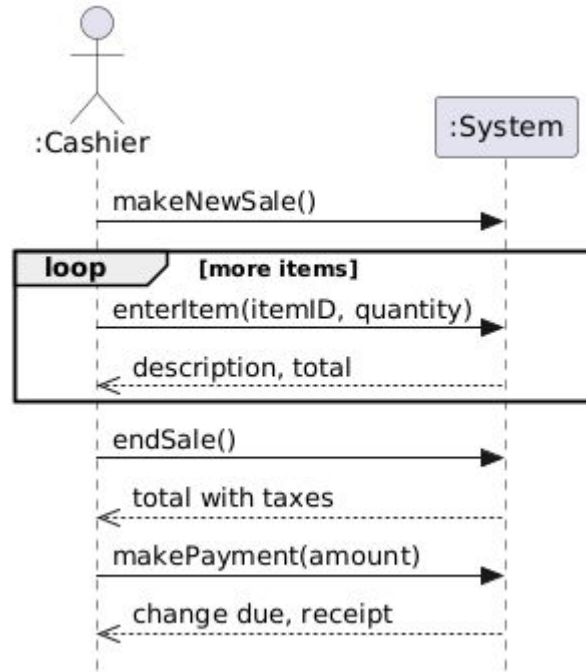
1. Customer arrives at a POS checkout with goods and/or services to purchase.
2. Cashier starts a new sale.
3. Cashier enters item identifier.
4. System records sale line item and presents item description, price, and running total.
Cashier repeats steps 3-4 until indicates done.
5. System presents total with taxes calculated.
6. Cashier tells Customer the total, and asks for payment.
7. Customer pays and System handles payment.

...



Input: What operations are needed?

Example: NextGenPos Input: SSD for Process Sale



These input system events
invoke system operations

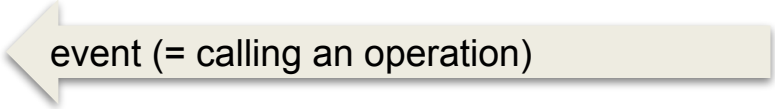
the system event `enterItem`
invokes a system operation
called `enterItem` and so forth

this is the same as in OO
programming when we say the
message `foo` invokes the method
`foo`

**Note: Difference between system events and
system operations**

Operation vs Event : in java code

```
Class Test{  
    SecondClass two = new SecondClass();  
  
    two.doSomethingWithData();  
  
}
```



event (= calling an operation)

```
Class SecondClass{  
  
    void doSomethingWithData(){  
        if time=today()  
            then do this  
        else  
            do that  
        end  
        ;  
    }  
}
```

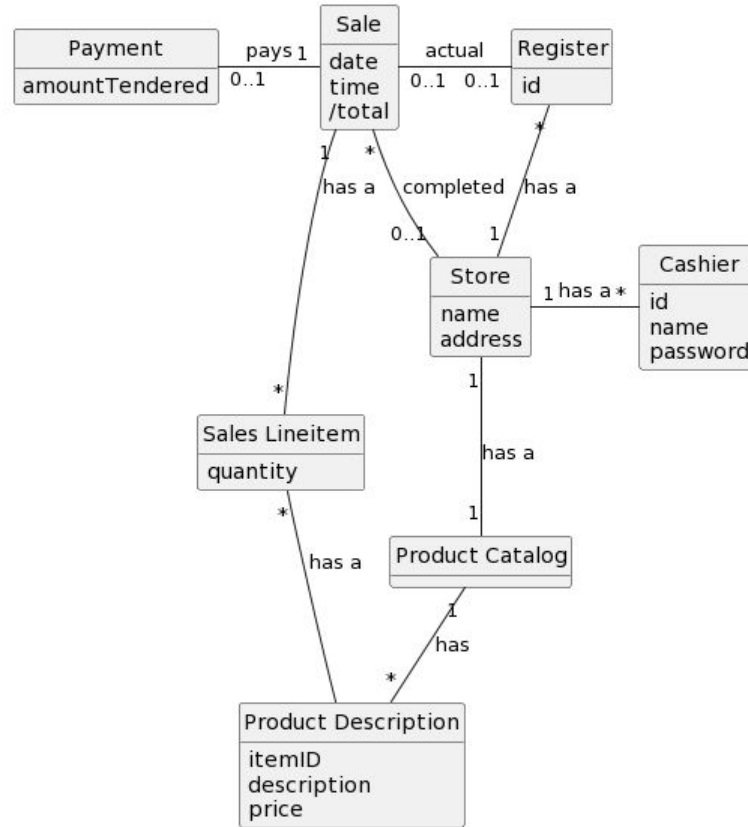


operation

Operation Contracts: Don't forget

- Operation Contracts are defined for System Operations
 - i.e. The operations (think: methods) that the system as a whole offers
 - So the system is still a 'black box' at this point of analysis (we don't know how the operations work)
- The System Sequence Diagrams shows system events
 - ie. The SSD's messages
- System operations handle the system events
- **When is it useful to make an OC?**
 - a) Does the operation in the system cause any changes when we consult the domain model?
 - b) Is the operation complex?

Example: NextGenPos Input: Domain Model



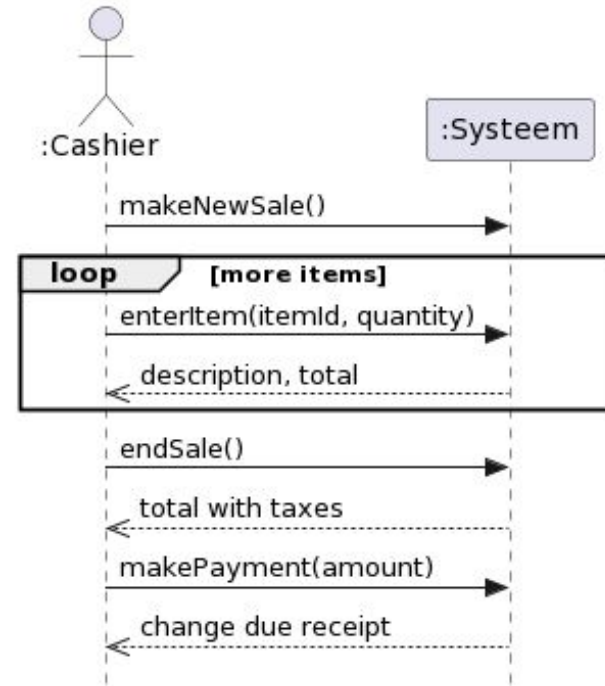
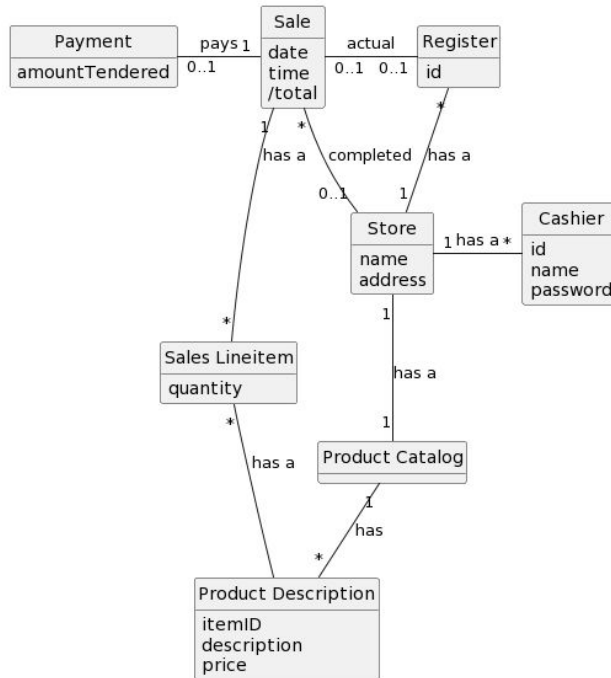
Which concepts DO/DO NOT participate in the UC: Process Sale?

Post Conditions

- Main part of the OC
- Contains changes to the state of the Domain Model
- Always check the following categories
 - Create / delete instances
 - Making/breaking associations
 - Change attribute values
- Write in past tense (passive)
 - A sale was created
- Remember: when creating an instance, associations usually also need to be made.

Example: POS

- Given: Domain Model + SSD
- Wanted: 4x Operation Contract



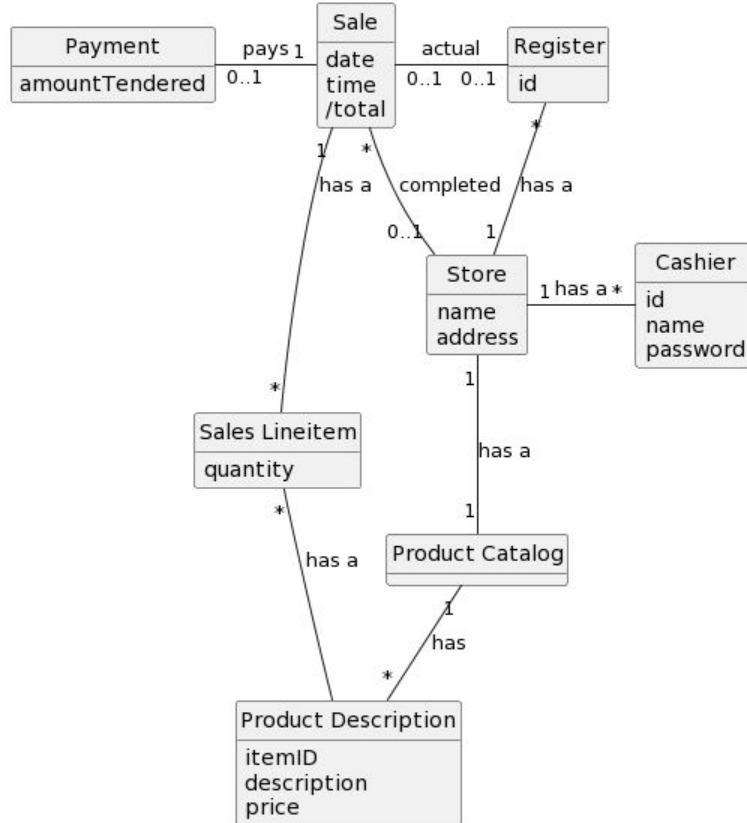
Example: POS / CO1 : makeNewSale

Operation: makeNewSale()

Cross References: ?

Preconditions: ?

Postconditions: ?



Example: POS / CO1 : makeNewSale

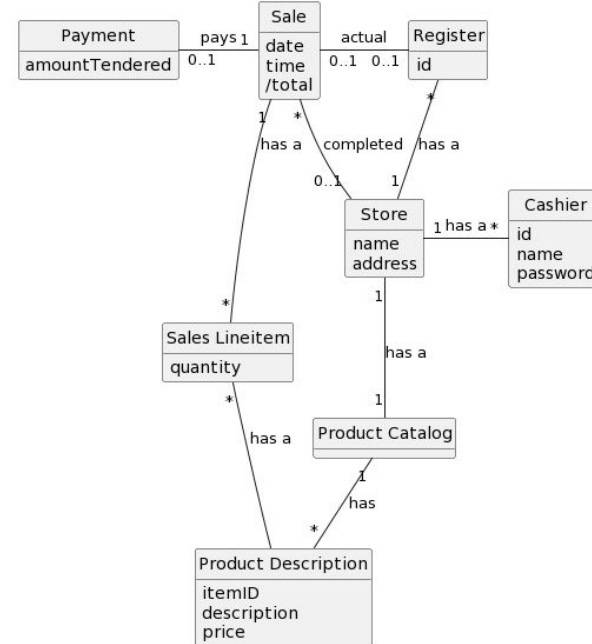
Operation: makeNewSale()

Cross References: Use Cases: Process Sale

Preconditions: A Register instance r existed

Postconditions:

- A Sale instance s was created
- s was linked with the Register instance r
- Attributes of s were initialized.



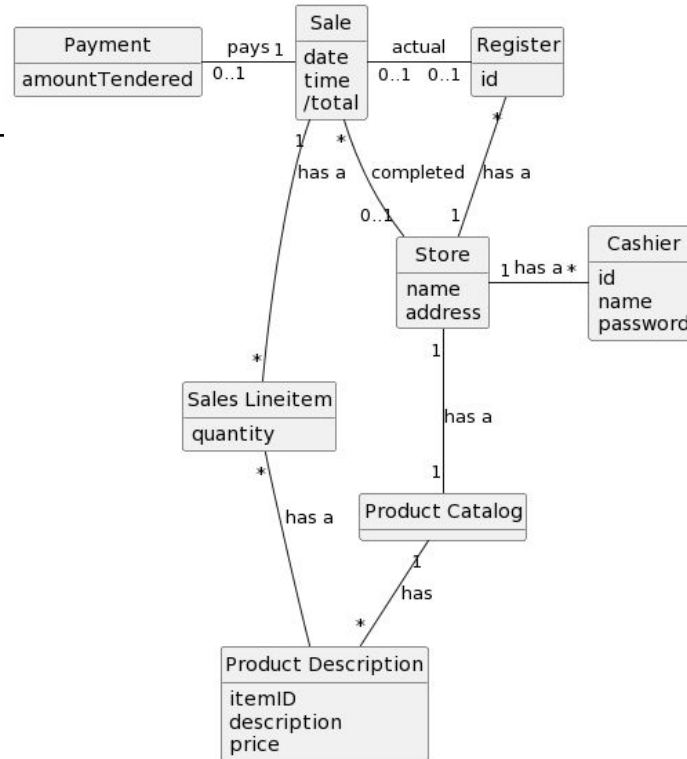
Example: POS / CO2 : enterItem

Operation: enterItem(itemID, quantity)

Cross References: ?

Preconditions: ?

Postconditions: ?



Example: POS / CO2 : enterItem

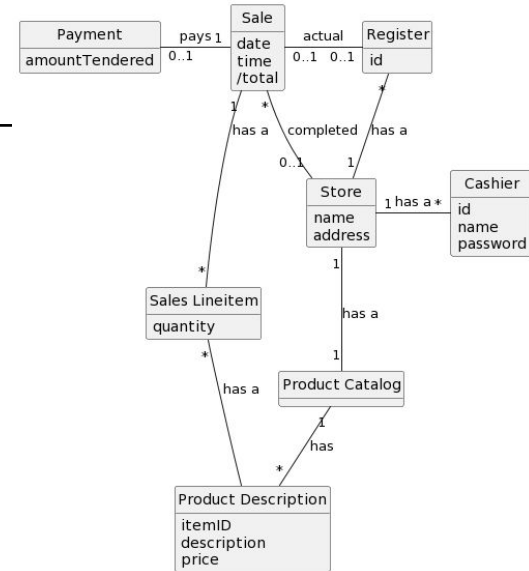
Operation: enterItem(itemID, quantity)

Cross References: Use Cases: Process Sale

Preconditions: A sale was started/created
ProductDescriptions are available

Postconditions:

- A SalesLineItem instance sli was created
- sli was associated with current Sale s
- sli was associated with ProdDescription
(use itemID from args to match the correct ProdDescription)
- Sli.quantity was set
(use quantity from args)



Example: POS / C03 : endSale

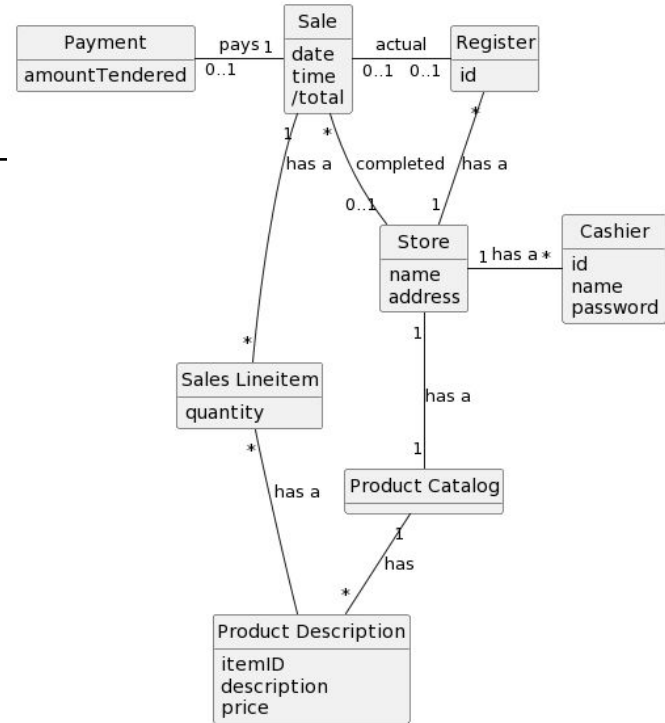
Operation: endSale()

Cross References: Use Cases: Process Sale

Preconditions: A sale is started/created

Postconditions:

- <no new instances>
- <no new associations>
- ??? How to track endSale??



Example: POS / CO3 : endSale

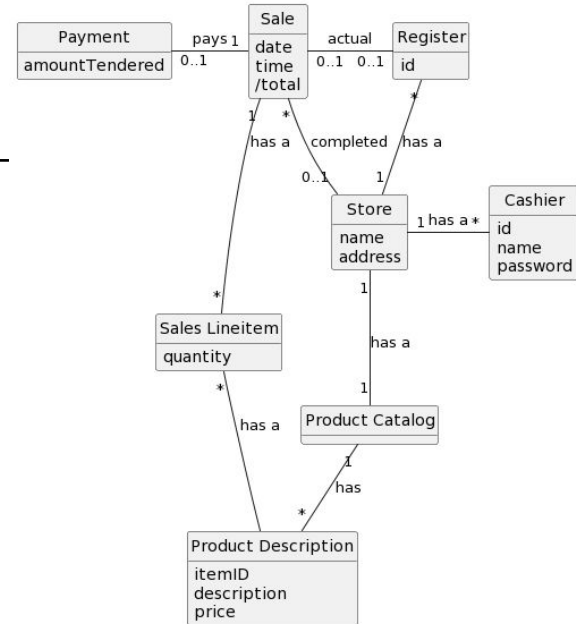
Operation: endSale()

Cross References: Use Cases: Process Sale

Preconditions: A sale is started/created

Postconditions:

- <no new instances>
- <no new associations>
- Sale.isComplete was set to true ?
new attribute needed in Domain Model ?
- Sale.total was filled in ?



These possible attribute changes are on the border of 'design/analysis' choices

Example: POS / CO4 : makePayment

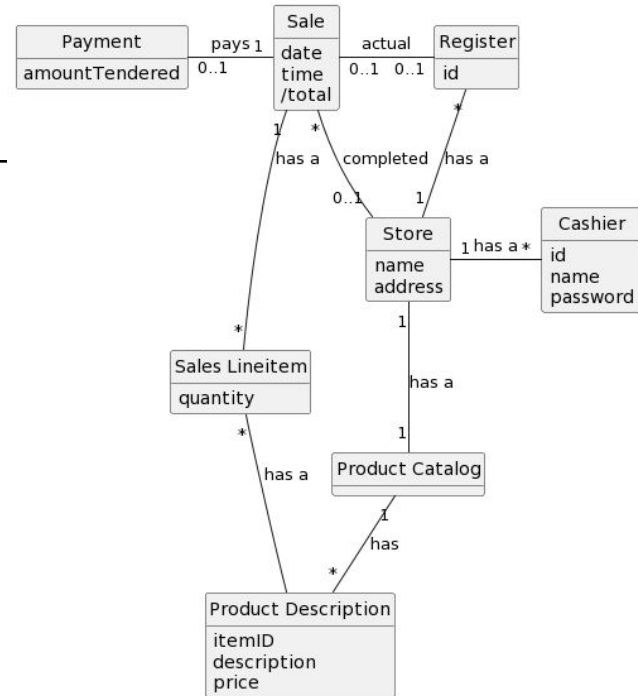
Operation: makePayment(amount)

Cross References: Use Cases: Process Sale

Preconditions: A sale is started/created

Postconditions:

- Which new instances?
- Which new associations?
- How to track sale was done?



Example: POS / CO4 : makePayment

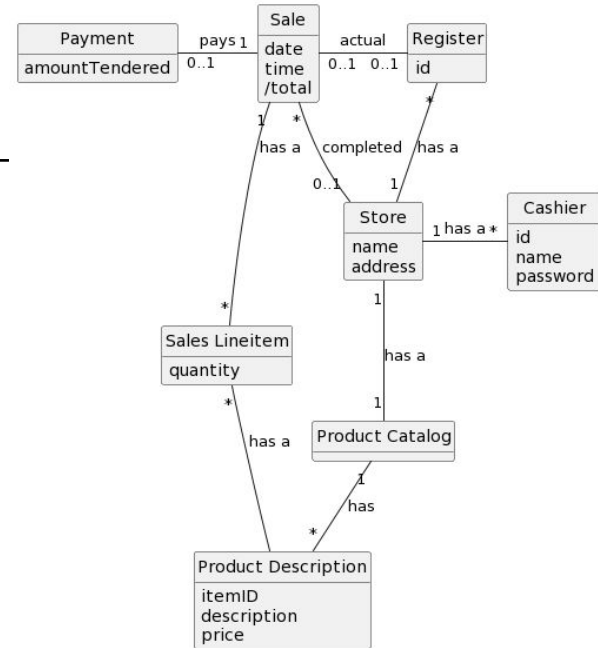
Operation: makePayment(amount)

Cross References: Use Cases: Process Sale

Preconditions: A sale is started/created

Postconditions:

- Payment instance *p* was created
- *p* was associated with current Sale *s*
- Sale *s* was associated to Store
- *p.amountTendered* was set
(use amount from args)
- Sale *s* was disassociated from Register



Method: How to draw up an Operation Contract (OC)?

1. **Use the template ([link](#))**
2. **Determine which operations are in one UC/SSD**
 - All solid arrows in SSD = input messages
 - Operations have the same names as input messages, and contain the responses to this query
3. **Determine postconditions:**
 - We only report system changes in OC postconditions ($\Leftrightarrow$ UC postconditions)
 1. An OC focuses on the system, not the UC process
 2. For example: "Membership was checked" is a process/UC result, but is not reported in OC
 - Three types of postconditions:
 1. Instance creations and deletions
 - Form: "An instance x of Xyz was created/deleted"
 2. Link creations and deletions
 - Form: "Instance x was linked/unlinked to instance y of Xyz"
 3. Changes in attribute values
 - Form: "x.attributename got the value \"...\" "
4. **Determine preconditions:**
 - Determine the preconditions based on the postconditions: what must exist for the operation to be performed correctly?
 - We only report system status in OC preconditions ($\Leftrightarrow$ UC preconditions)
 - For example: which objects are present, which links are present
 - An OC focuses on the system, a UC on the process
5. **Only include alternatives when specifically asked ($\Leftrightarrow$ UC descriptions)**

CRUD as a control

- **CRUD = Create, Read, Update, Delete**
- **These are the 4 basic types of data storage functions**
- **So, what types of postal conditions should you include in an OC:**

	Instance	Link	Attribute value
Create	YES	YES	
Read			
Update			YES
Delete	YES	YES	

- **Note: no read data**

Template Operation Contract



11.2

Contract OC# :	
Operation:	
Cross-reference:	Use Case(s) :
Precondition:	Get instance within scope of UC (form: "Instance x of Xyz was present")
Postconditions:	Instance creations and deletions (form: "Instance x of Xyz was created/deleted") •
	Link creations and deletions (form: "Instance x of Xyz was linked/unlinked from instance y of Xyz") •
	Changes in attribute values (form: "x.attributeName was given the value [...] ") •

Recurring Example: Monopoly

Purpose of the Exercise

This exercise is an ongoing case study in Conceptual Thinking. Each week, we apply what we've learned of Monopoly.

Why do we choose Monopoly?

- Everyone knows it, or almost everyone
- It contains rich structures: players, money, rules, actions, spaces, cards
- It provides an ideal context for modeling object-oriented systems
- It is tangible and recognizable, making abstract concepts concrete



Recurring Example: Monopoly

Learning objective

Students understand how theory is translated into a working conceptual model by:

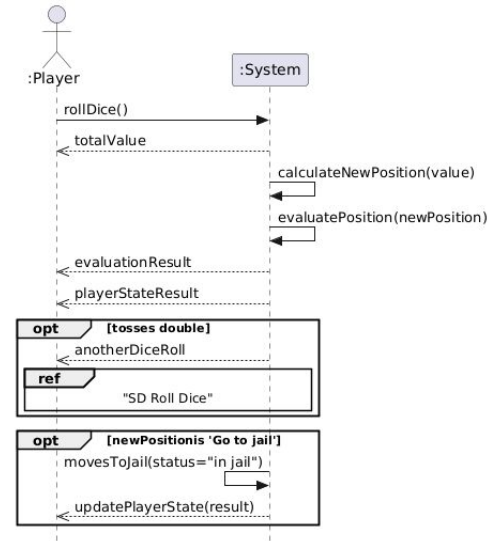
- Working iteratively with one consistent case
- Apply different analysis methods (use cases, class diagrams, ...)
- Gain insight into the relationship between models and system behavior

Quick Refresher: How Does Monopoly Work Again?

- 2–6 players move their pawn around the board with a roll of 2 dice
- Players buy properties they land on
- Other players pay rent when they land on someone else's property
- There are special subjects such as Chance, Community Fund, Tax, Prison, etc.
- Players can take out mortgages, trade properties, or go bankrupt
- Goal: Become the richest player by bankrupting others

Use case & SSD (1/3)

Use case 1	Roll Dice
Name	Roll Dice
Goal	Move the player based on a die roll
Actors	Player
Preconditions	- It's the player's turn - Game is active
Postconditions	- Player has been moved - New position is being evaluated
Trigger	Player clicks on 'Roll Dice'
Main success path	1. Player chooses 'Roll Dice' 2. Dice generates a value 3. Game counts total 4. Game shows total of dice roll 5. Game calculates new position 6. Player pawn is moved 7. New position is being evaluated 8. Evaluation and state are shown to player
Alternative paths	- A1: Player throws double → extra turn - A2: Player lands on 'Go To Jail' → moves to Jail, status = "in jail"



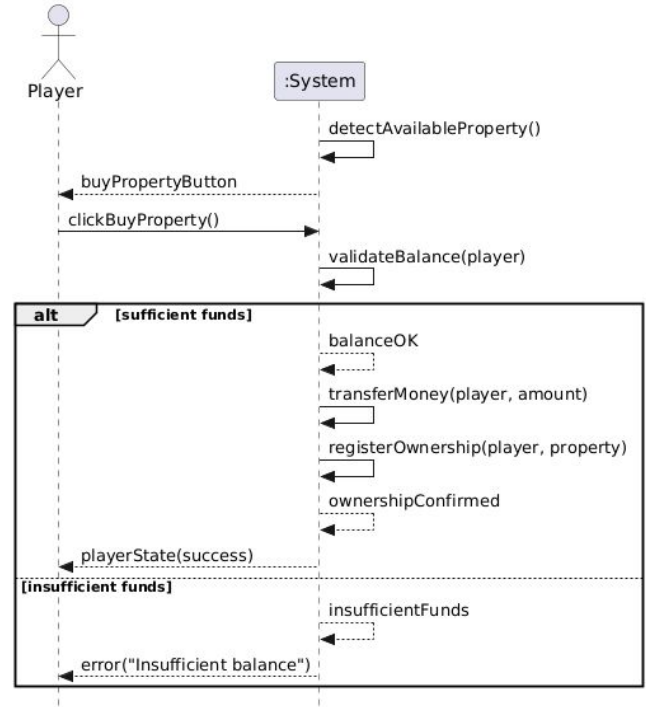
Note: System does not call itself in an SSD

Use case & SSD (2/3)

Use case 2

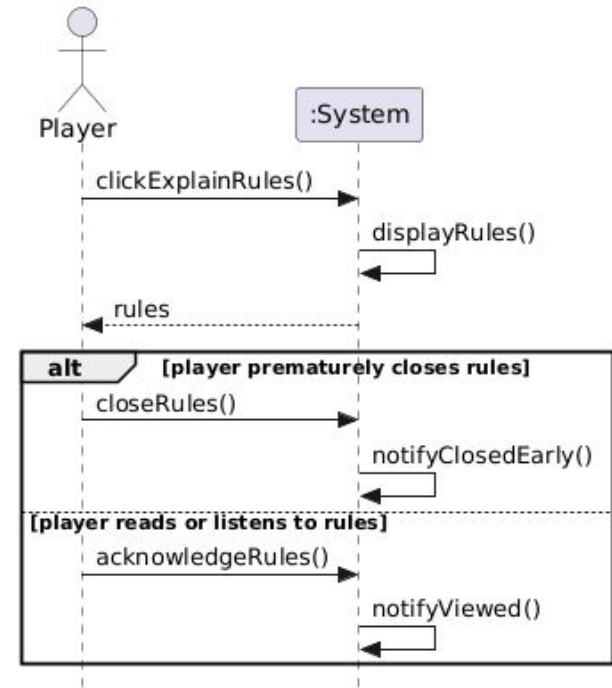
Buy Property

Name	Buy Property
Goal	Let player buy a vacant property
Actors	Player
Preconditions	- Player stands on unoccupied property - Player has enough money
Postconditions	- Property added to player - Player's balance has decreased
Trigger	Player chooses the option 'Buy Property'
Main success path	1. Game detects available ownership 2. The 'Buy Property' button appears 3. Player chooses 'Buy Property' 4. Bank validates balance 5. Money is transferred 6. Ownership is registered
Alternative paths	- A1: Player has insufficient balance → purchase declined - A2: Player refuses purchase → property remains unowned



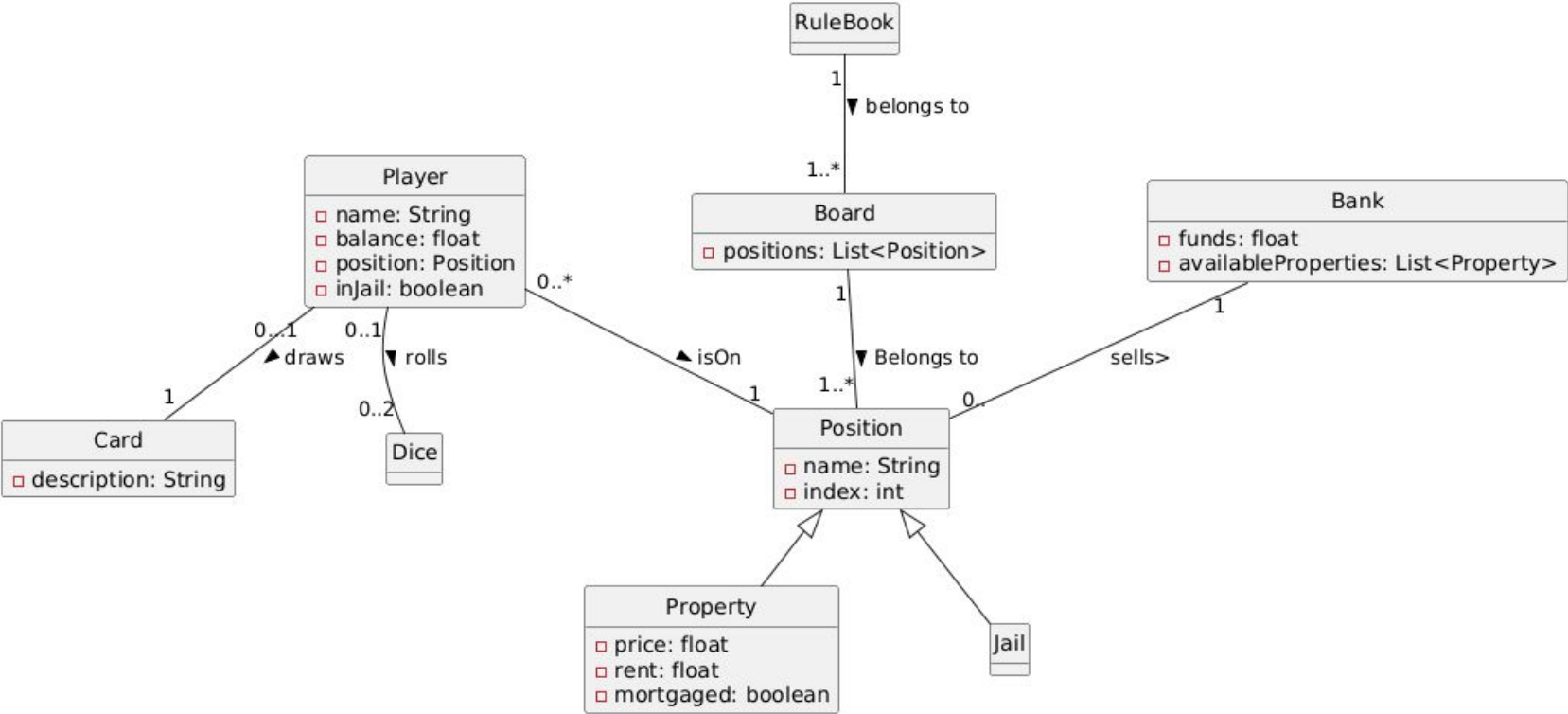
Use case & SSD (3/3)

Use Case 3	
Name	Explain Rules
Goal	Show game rules to new players
Actors	Game Organizer
Preconditions	- Game hasn't started yet
Postconditions	- Players have had access to game rules
Trigger	Organizer chooses option 'Explain Rules'
Main success path	1. Organizer selects 'Explain Rules' 2. System shows rules 3. Players read or listen to explanation
Alternative paths	- A1: Players close lines prematurely - A2: Organizer skips lines



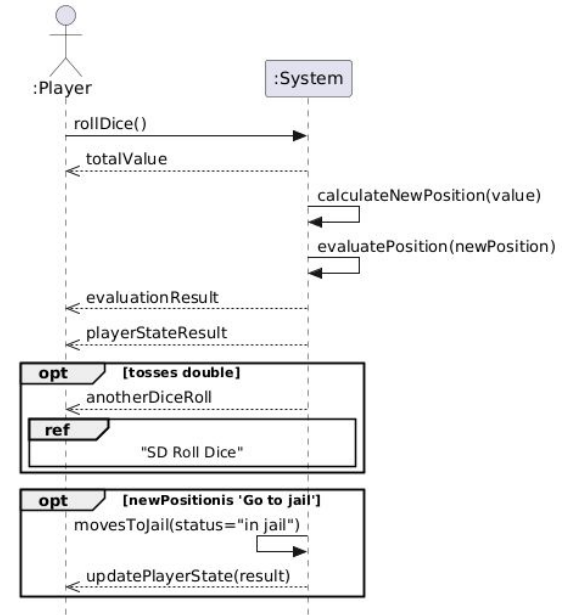
Conceptual Class Diagram

Conceptual Class Diagram - Monopoly



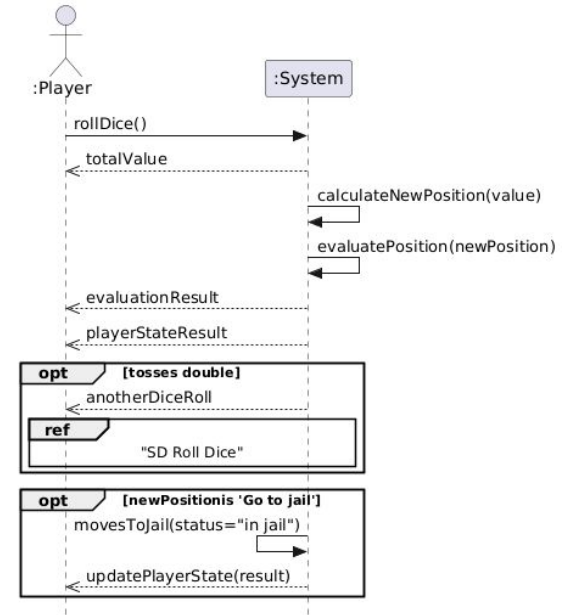
OC: UC Roll Dice (1/4)

Contract OC1	
Operation	rollDice()
Cross-reference	Use case 1 - Roll dice
Preconditions	An instance p of the class Player exists An instance pos of the class Position exists p and pos are linked
Postconditions	Create an instance d of the class Dice
	Link d to p
	N/A



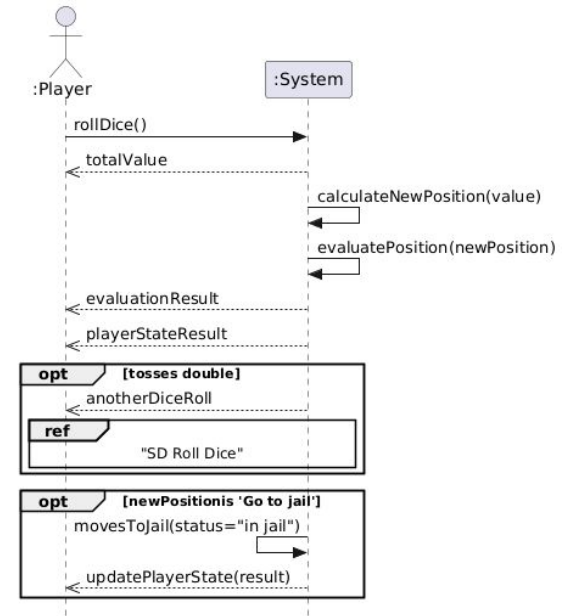
OC: UC Roll Dice (2/4)

Contract OC2	
Operation	calculateNewPosition(value)
Cross-reference	Use case 1 - Roll dice
Preconditions	An instance b of the class Board exists An instance p of the class Player exists An instance pos of the class Position exists b and pos are linked p and pos are linked
Postconditions	N/A
	N/A
	Update position



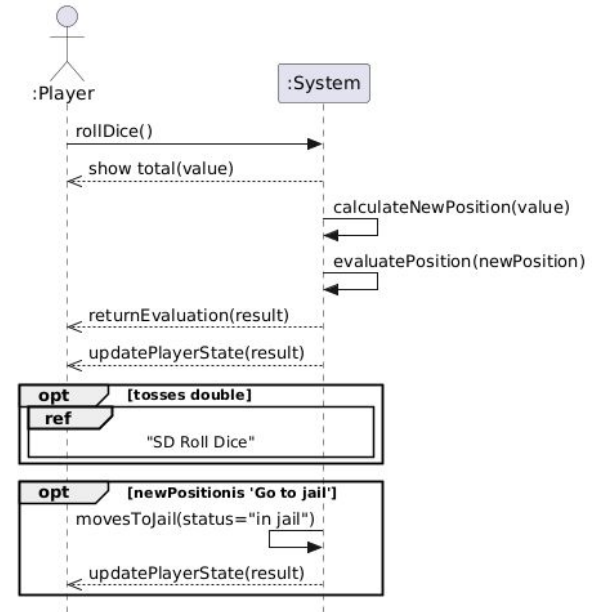
OC: UC Roll Dice (3/4)

Contract OC3	
Operation	evaluatePosition(newPosition)
Cross-reference	Use case 1 - Roll dice
Preconditions	An instance b of the class Board exists An instance p of the class Player exists An instance pos of the class Position exists b and pos are linked p and pos are linked
Postconditions	N/A
	N/A
	N/A



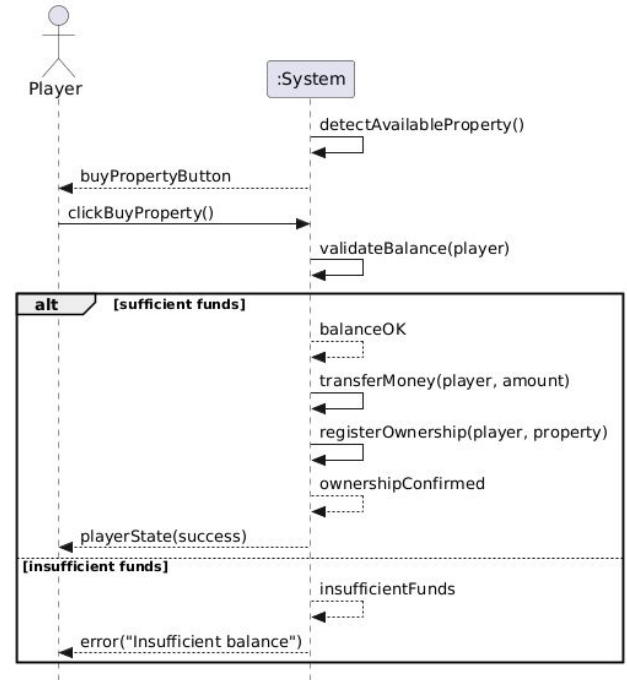
OC: UC Roll Dice (4/4)

Contract OC4	
Operation	movesToJail(injail=true)
Cross-reference	Use case 1 - Toss dice
Preconditions	An instance b of the class Board exists An instance p of the class Player exists An instance pos of the class Position exists b and pos are linked p and pos are linked
Postconditions	N/A
	N/A
	Update injail



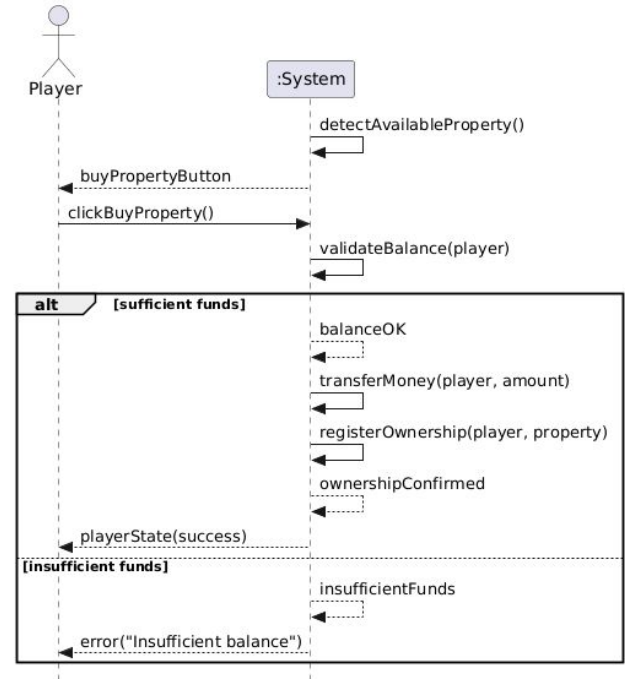
OC: Buy property (1/5)

Contract OC1	
Operation	detectAvailableProperty()
Cross-reference	Use case 2 - Buy property
Preconditions	An instance p of the class Player exists An instance pos of the class Position exists An instance prop of the class Property exists An instance b of the class Bank exists p and pos are linked prop and pos are linked b and pos are linked
Postconditions	N/A
	N/A
	N/A



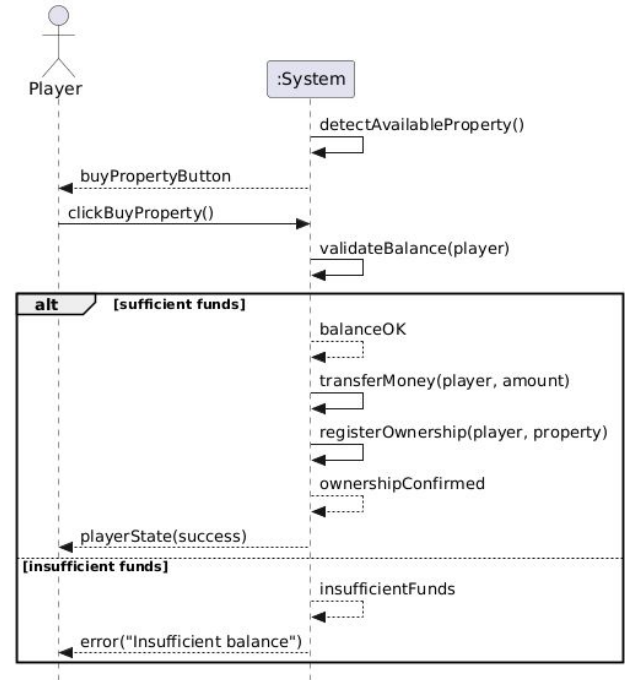
OC: Buy property (2/5)

Contract OC2	
Operation	clickBuyProperty()
Cross-reference	Use case 2 - Buy property
Preconditions	An instance p of the class Player exists An instance pos of the class Position exists An instance prop of the class Property exists An instance b of the class Bank exists p and pos are linked prop and pos are linked b and pos are linked
Postconditions	N/A
	N/A
	N/A



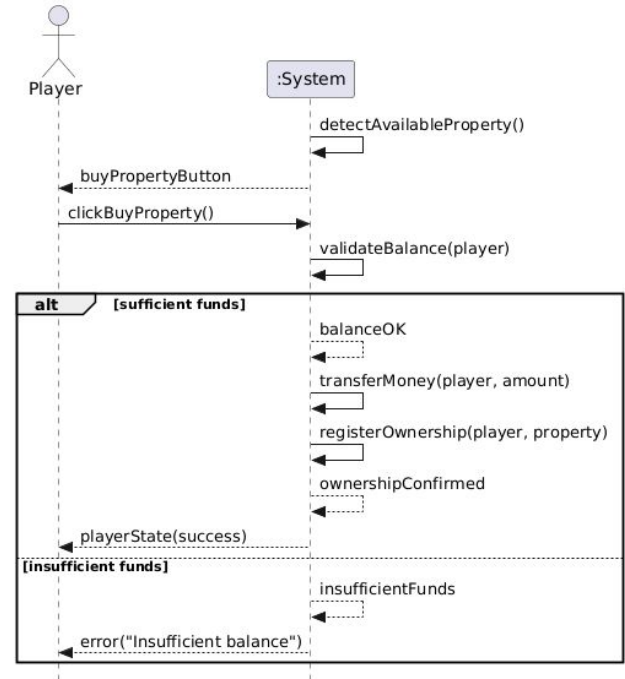
OC: Buy property (3/5)

Contract OC3	
Operation	validateBalance(player)
Cross-reference	Use case 2 - Buy property
Preconditions	An instance p of the class Player exists An instance pos of the class Position exists An instance prop of the class Property exists An instance b of the class Bank exists p and pos are linked prop and pos are linked b and pos are linked
Postconditions	N/A
	N/A
	N/A



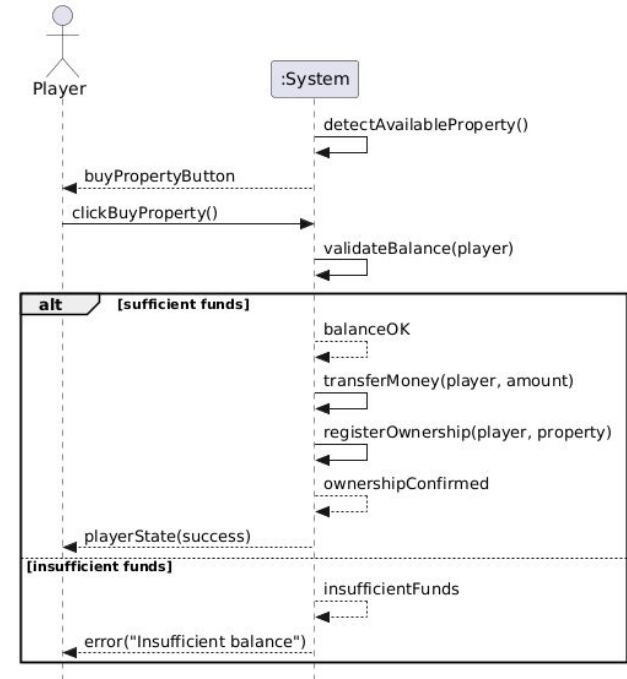
OC: Buy property (4/5)

Contract OC4	
Operation	transferMoney(player, amount)
Cross-reference	Use case 2 - Buy property
Preconditions	An instance p of the class Player exists An instance pos of the class Position exists An instance prop of the class Property exists An instance b of the class Bank exists p and pos are linked prop and pos are linked b and pos are linked
Postconditions	N/A
	N/A
	Update balance



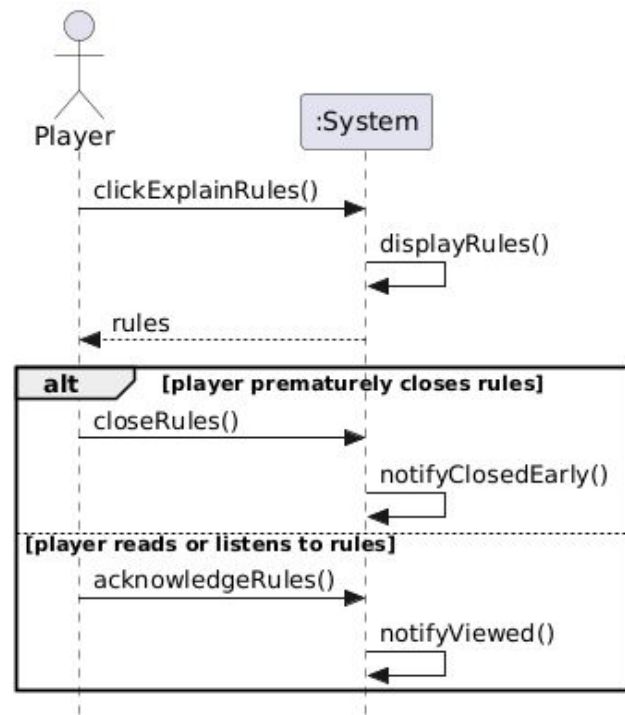
OC: Buy property (5/5)

Contract OC5	
Operation	registerOwnership(player, property)
Cross-reference	Use case 2 - Buy property
Preconditions	An instance p of the class Player exists An instance pos of the class Position exists An instance prop of the class Property exists An instance b of the class Bank exists p and pos are linked prop and pos are linked b and pos are linked
Postconditions	N/A
	N/A
	Update availableProperties



OC: Explain rules

Contract OC1	
Operation	displayRules()
Cross-reference	Use case 3 - Explain rules
Preconditions	An instance rb of the class RuleBook exists An instance b of the class Board exists rb and b are linked
Postconditions	N/A
	N/A
	N/A



Questions?